

Prototype - Registry | Design pattern

Prototype Design Pattern

{ → Given an object of a class
→ We need to create copy of the object }

Client {

psvm() {

⇒ Student st = _____;

Student st2 = copy of st;

}

}

Solⁿ: Use copy constructor

psvm() {

⇒ Student (st1) = getStudent();

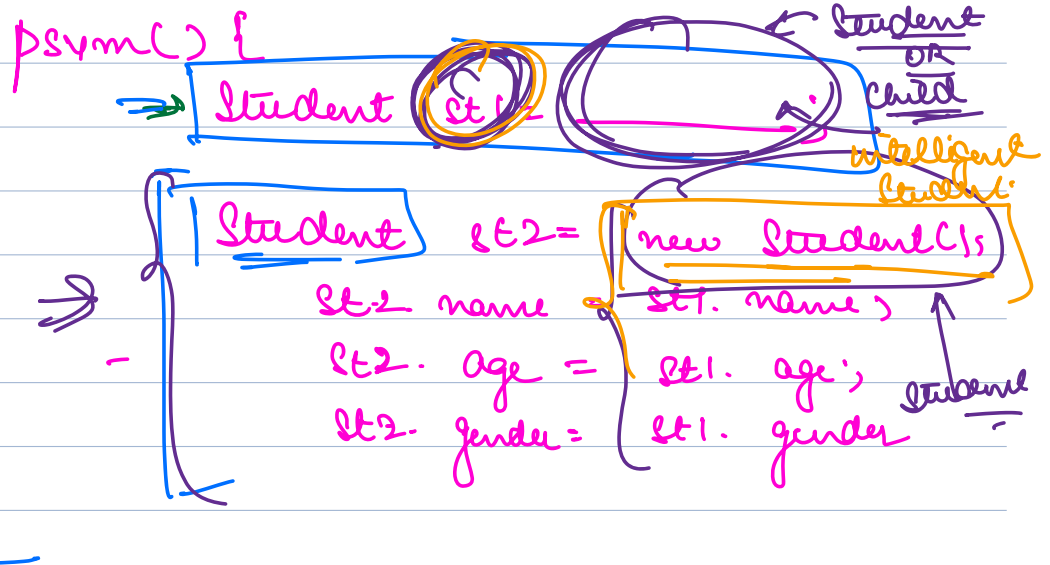
Student st2 = new Student (st1);

}

Implementation detail → ✓
OCP violation ⇒ X
private attr ⇒ ✓

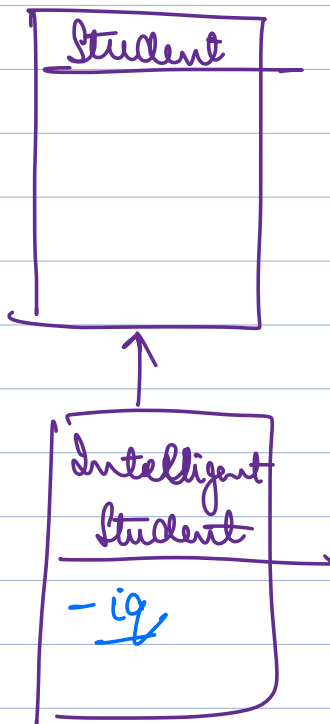
Solⁿ 2

Copy attr manually.

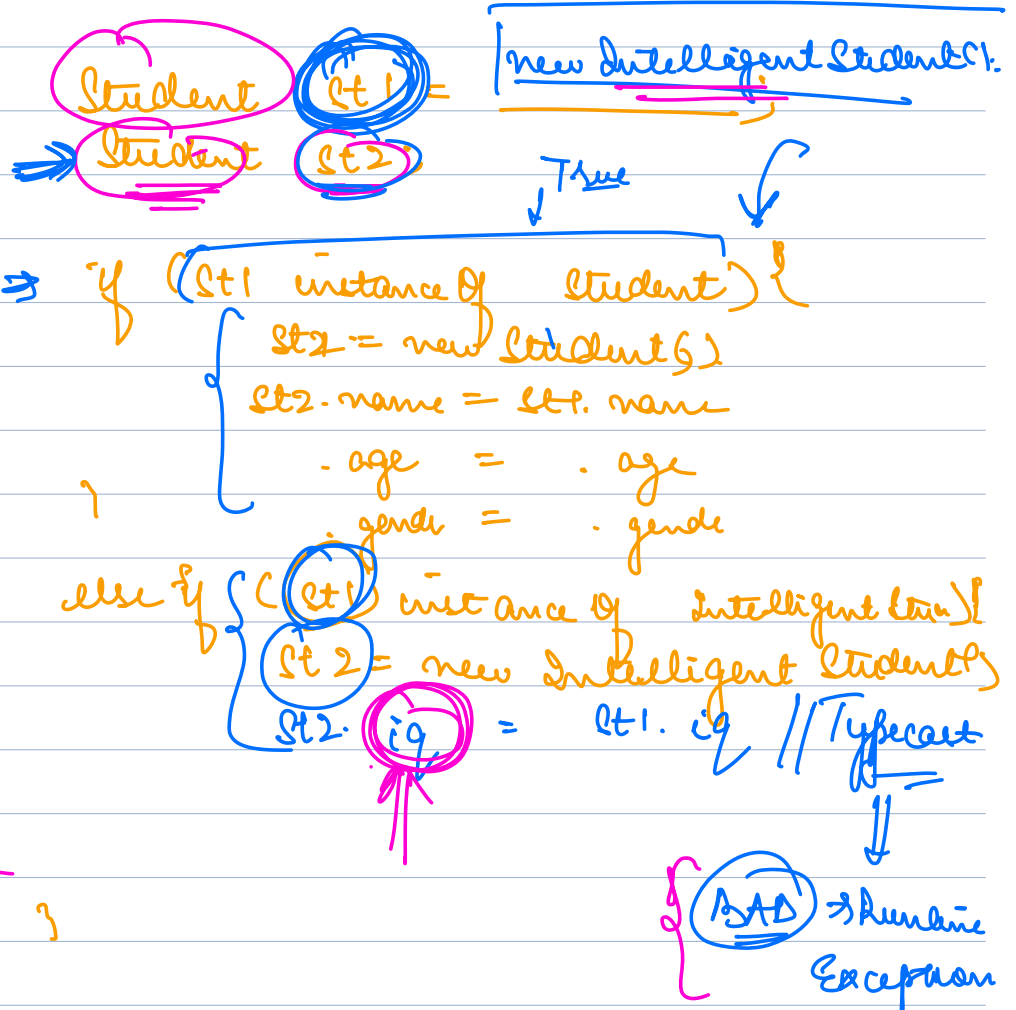


Problems:

- ① If private attr not even able to copy it.
- ② Client will need to know everything about `Student`
⇒ Tight Coupling



psvm() {



- ① Not always possible $\Rightarrow$ attr may be private
- ② Implementation detail is exposed of classes.
- ③ Child clones!!! $\Rightarrow$ violate OCP

```
Student st1 = get Student();
```

```
get Student() {  
    if http. return. value = i  
        return new Intelligent Student;  
    else  
        return new Student();  
}
```

```
i = random() % 2  
if (i == 1) {
```

```
    Create Copy ( Object o ) {  
        Object o = new Object();  
    }
```

Diagram illustrating object creation and copying:

- A box labeled "Create Copy (Object o)" contains the code "Object o = new Object();".
- Arrows point from the box to three labels: "Student", "IS", and "Mentor".
- A question mark "?" is placed above the box.

Summary

- ① object of a class
- ② return a copy of that object

Client {

psvm() {

Object o = new Student().

Object o1 =

}

}

- Sol^{ns}
- ① copy attrs manually.
 - ② call copy constructor

Client {

psvm() {

Student st1;

SRP {

→ if st1 instance of Intelligent Student {
 // st2 = new Intelligent Student(st1);

→ , else if {
 // st2 = new Student(st1);

}

}

}

```

Student {
  {
    attr
  }
  {
    methods
  }
}

```

```

Client {
  psvm() {
    Student st1 = IS;
    Student st2 = st1.copy();
  }
}

```

```

=> Student copy() {
  Student st1 = new Student();
  st1.name = this.name;
  st1.age = this.age;
  st1.gender = this.gender;
  return st1;
}

```

Intelligent Student extends Student {
 if.

doSomething();

```

{
  Intelligent Student copy() {
  }
}

```

⇒ Because of method overriding property of polymorphism, OCP is solved!!

BUTS

① Every child class must override copy.

Intelligent Student

copy()

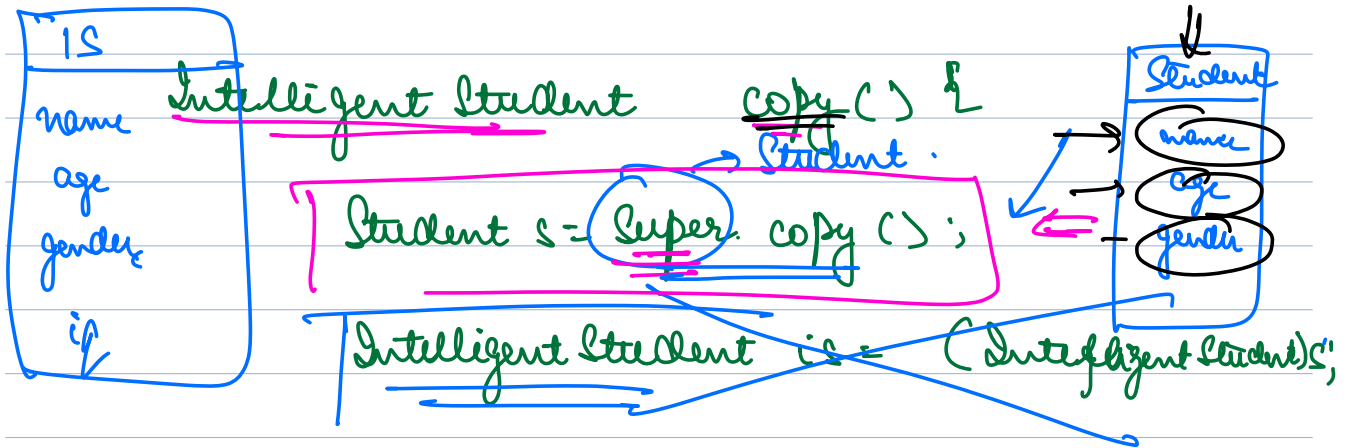
```
Intelligent Student i1 = new ISC();  
i1. id = this. id;  
i1. name = this. name;
```

if name
is private
how want
work.

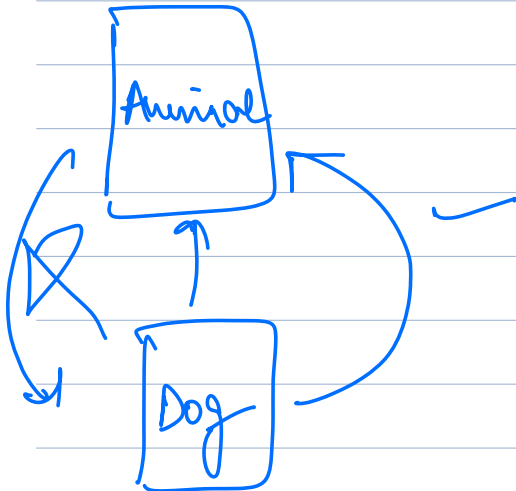
X : C

}

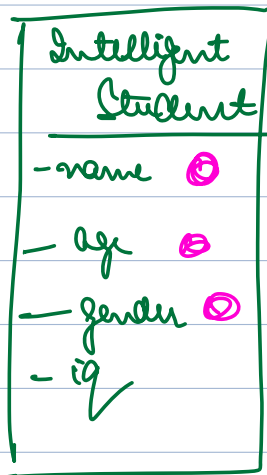
}



?



Intelligent Student extends Student {



Intelligent Student (Intelligent Student is) {
 => super (is);

this . iq = 180;

}

Class Intelligent Student extends Student {

 Intelligent Student (Intelligent Student other) {
 super (other);

 -> this . iq = other . iq;

 }

 Intelligent Student copy () {
 return new IS (this);

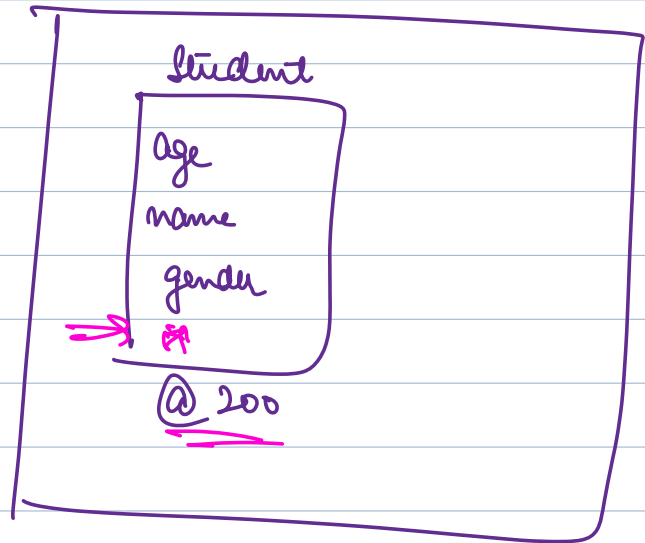
}

}

```

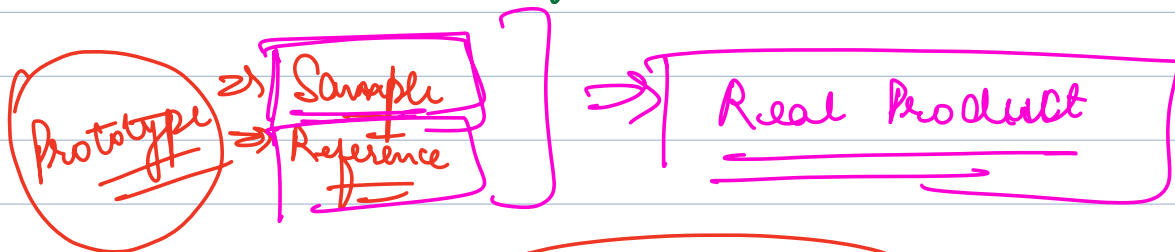
{
  is = @ 200
  (is) iq = 400
}

```



Why might someone want to create a copy?

① avoid modifyⁿ to original object



We want to use Google Search's API + many more APIs

wherever you want to use Google search:

```

HTTPReq r = new HTTPRequest()
r.set Path( "google.com/ search")
r.set Username( — )
r.set Password( — )
r.set Type (TextOnly Search)
r.set Query (q)
r.execute()

```

Image Search
Video Search

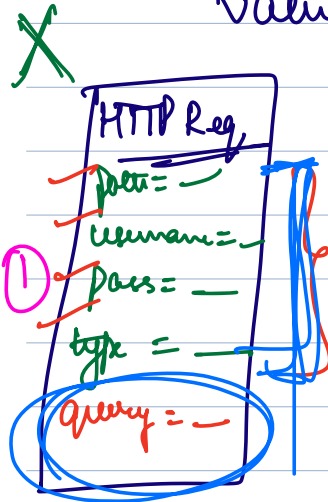
Only thing
to differ
for every
Search.

for every search req,
you create an object
of HTTPReq

Prototype Design Pattern

① → for commonly needed objects you store
a prototype of them which has base
values set.

(values that
will remain
common)



② whenever you need that obj,
you will create a copy from
prototype.

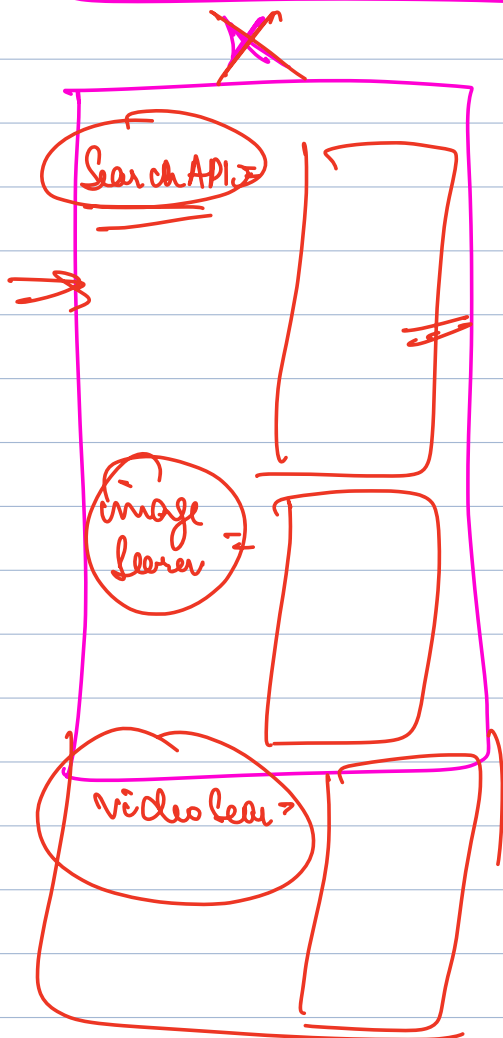
②
HTTPRequest ~~is~~ X. get ("Search Query")

• copy!

r.set Query (india)
r.execute()

HTTPReq r = X. get ("image search api")

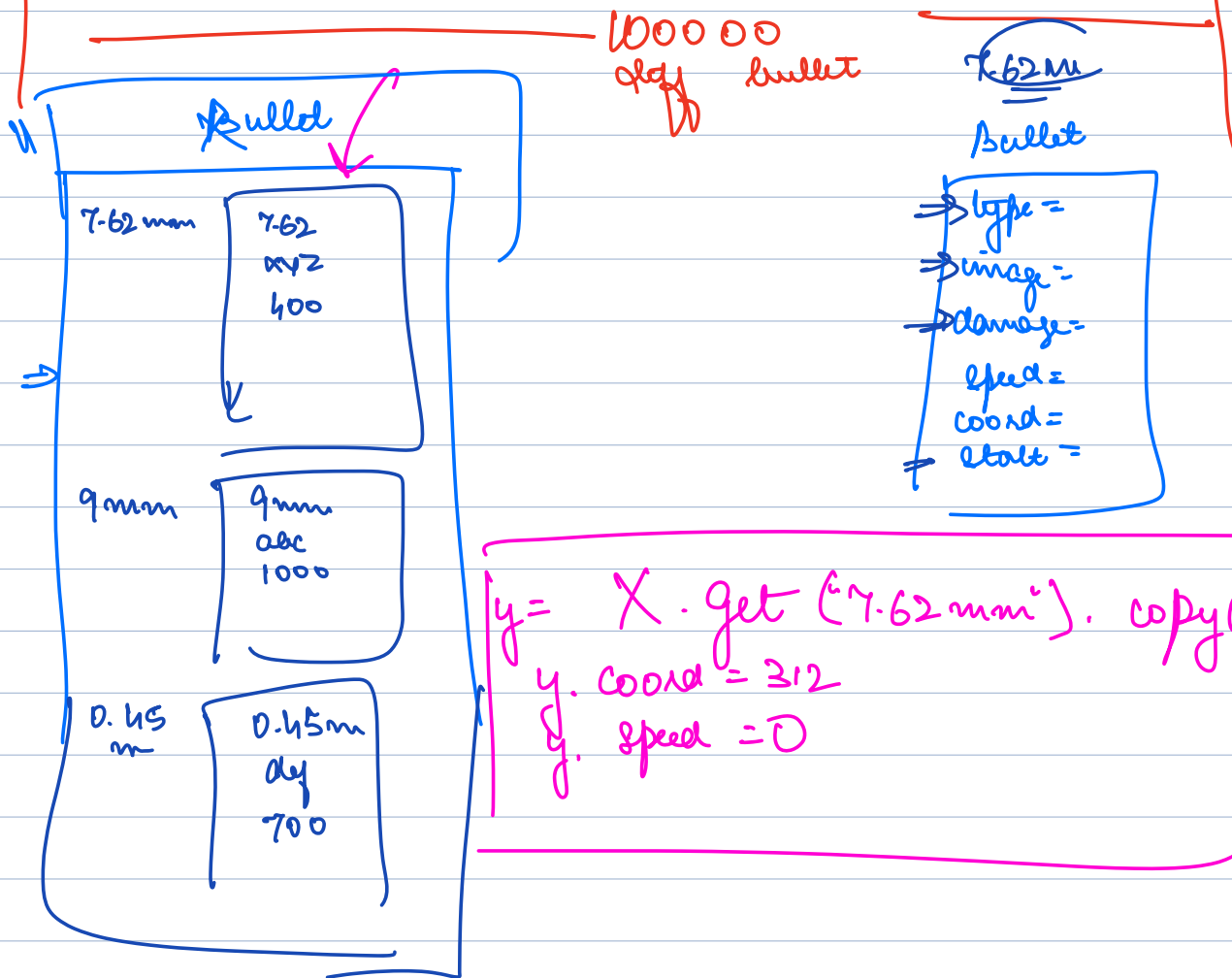
HTTPReq r = X. get ("image search")
r. query ("india")
r. execute()



~~X~~

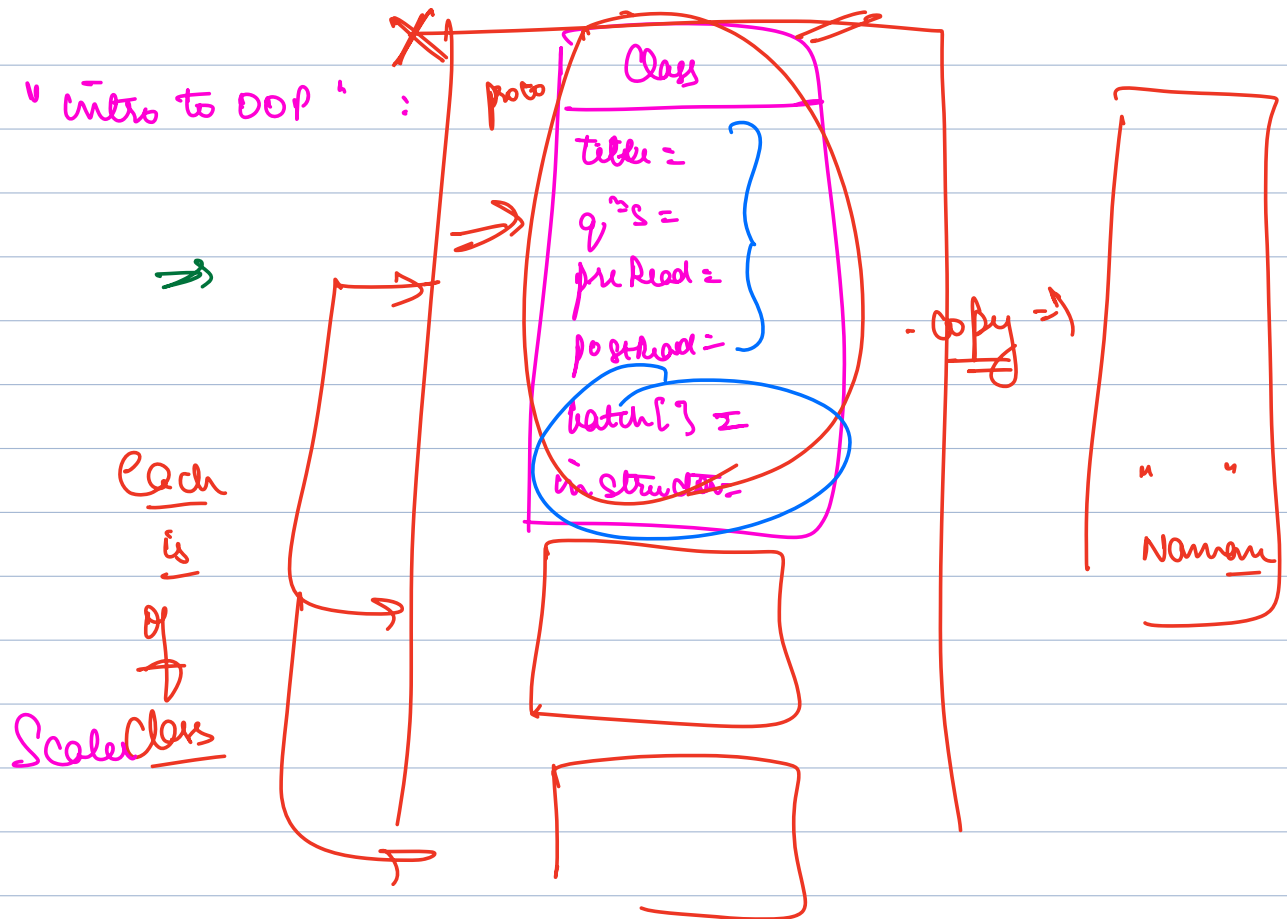
SearchQuery = select *
from { }
where { }

3 types of Bullet



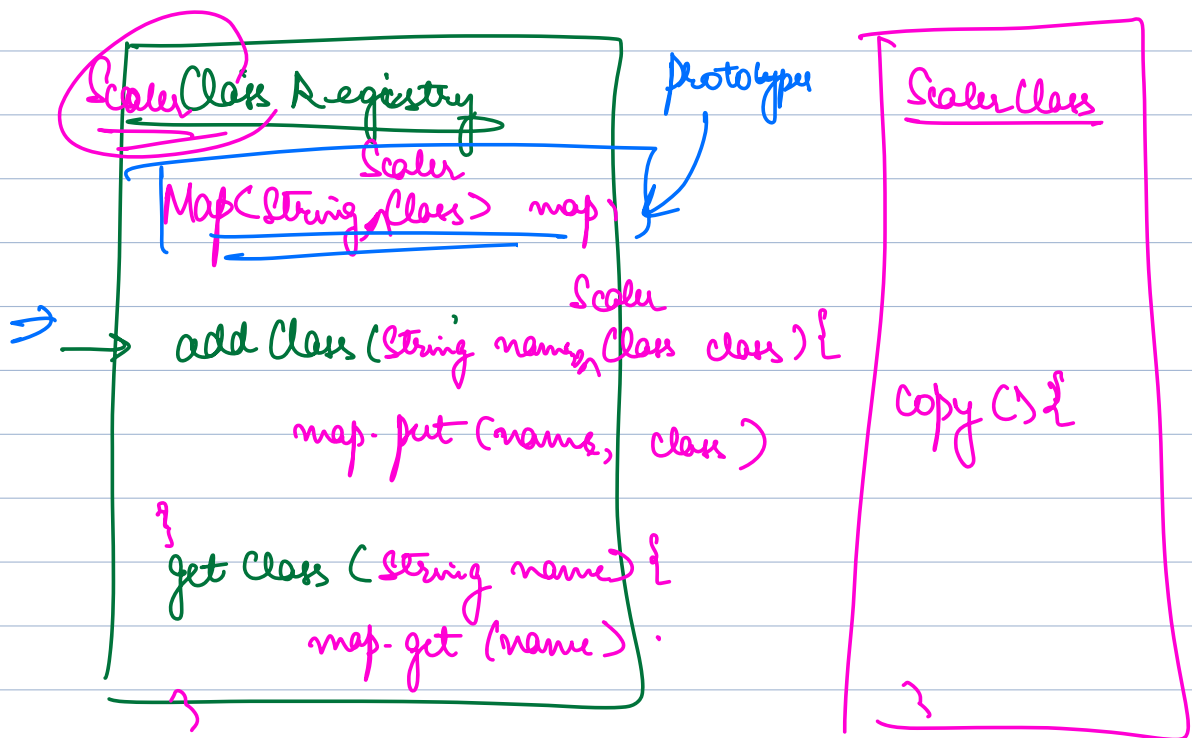
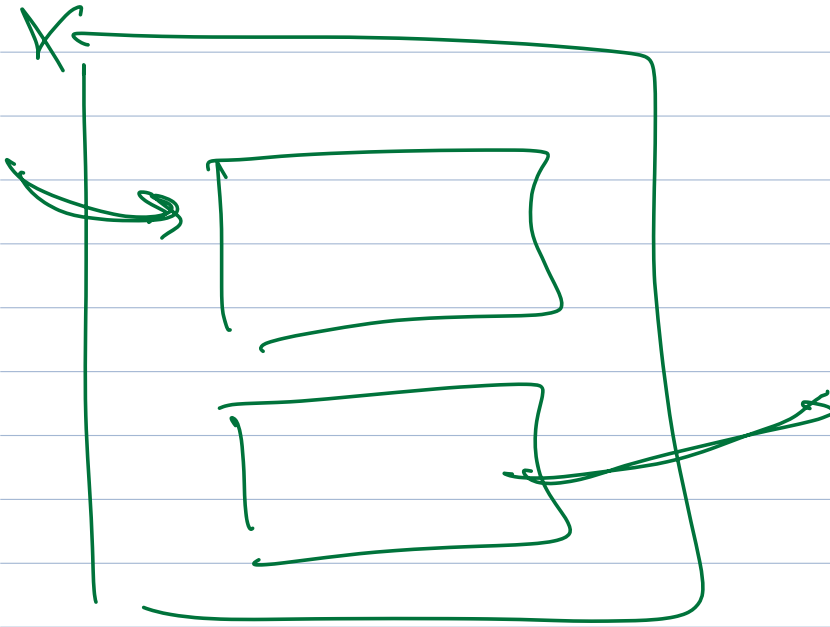
In Scala's codebase for each class a temp is already created

'Intro to OOP' :



⇒ Storing of common state of obj of that class

X ⇒ Registry ⇒ storing diff variations of a class



Main {

Map

```
psvm() {  
    Registry.initialize();  
    {  
        ScalarClass sc = ScalarClassReg.get  
            ("introToOOP");  
        • copy();  
    }  
}
```

sc.batch = "Apr 22 Intro";

sc.instructor = "Naman";

}

→ We must ensure across app^m only one
obj of Registry.

SUMMARY

why

→ Class for which objects are created from
common variation

eg: scalar → classes are from fixed
set of titles

pubg → bullets are from fixed
(set of type)

→ I put those fixed set in registry

→ Class must have copy() so that I can

{ Create diff obj from prototype

⇒ How to create a copy correctly

| copy() → copy constructor